FINAL PROJECT

2026-06-11

AUTOMATIC ANALYSIS AND GRADING OF UTML UML DIAGRAMS

Douwe Osinga
d.r.osinga@student.utwente.nl

Supervisor
dr. ir. Vadim Zaytsev
v.zaytsev@utwente.nl

Supervisor
dr. Nacir Bouali
n.bouali@utwente.nl

Faculty of Electrical Engineering,
Mathematics, and Computer Science

UNIVERSITY OF TWENTE.

ABSTRACT

During computer science studies, students are often required to submit diagrams. The grading of these diagrams is currently done by humans, resulting in a costly, lengthy, and error-prone process. In this paper, we investigate the theoretical feasibility of automatically grading diagrams, focusing on Unified Modelling Language diagrams and the UTML file format used by the University of Twente. Existing work shows that graph isomorphism algorithms which account for the use of synonyms and the presence of spelling mistakes provide the best results, but autograders utilising this strategy do not provide their source code. Based on these findings, we propose *Seshat*, an open-source, generic autograder that combines the aforementioned techniques and is capable of supporting arbitrary diagrams and file formats, with built-in support for UTML. In the final thesis, we realise *Seshat* and compare it to human grading for multiple UTML exam submission datasets.

1. INTRODUCTION

Add picture of UML diagram :)

Unified Modelling Language (UML) diagrams, introduced by the Object Management Group [1], play a significant role in computer science, as they allow for communicating software designs in a standardised format. During technical studies, students are often required to make UML diagrams for graded assignments or exams.

However, the grading of these diagrams is often a costly and lengthy process, involving multiple paid members of staff [2][1]. Additionally, this process is prone to grading inconsistencies due to various reasons [2], but mainly the inconsistency of human graders [2] [3]. M. Meadows *et al.* [3] pose two possible solutions to the problem of human grading: either "report the level of reliability associated with marks/grades, or find alternatives to [grading]." We propose a third alternative: finding alternatives to the grading *process*.

The (partial) automatisation of grading diagrams ('autograding') provides a grading paradigm that can both reduce the cost and time required for institutions and reduce the inherently present inconsistencies in human grading[2] [4] [5]. This could result in similar or superior performance compared to human grading in terms of **accuracy**, **grading transparency**, and **consistency**.

With *accuracy*, we mean the percentage of points assigned to a submission that are prescribed by the rubric for a particular excercise. With *consistency*, we mean both the extent to which similar grades are given to similar submissions and the difference between consecutive runs (i.e. determinism). With *grading transparency*, we mean the extent to which the reasoning for a particular grade is explained with regards to the rubric for the exercise or to the Intended Learning Objectives (ILOs) of a module. These properties are desirable in the grading process, as it means that students are graded in a way that reflects their performance (*accuracy*), allows them to see which parts they could improve for future assignments (*grading transparency*), and is minimally unfair (*consistency*).

Specifically for the implementation, we desire an autograder that can *link its grading to ILOs*, as described by the previous paragraph. Furthermore, built-in *UTML support* is a must, as it is the main file format the University of Twente uses. Finally, an *easily extensible* autograder would be ideal, since we might want to extend support to other file formats or alter the behaviour of the autograder later on.

1.1. Background

The idea of letting a computer program (partially) grade tests has been discussed in papers since the 70s [6, p.13], with some papers with implementations starting to appear around the 80s, primarily focused on grading the writing style of computer programs [7]. Interest in grading diagrams specifically seems to have started around the early 2000s [8] [9].

Diagrams themselves have multiple variants for different purposes. UML diagrams, for example, mainly serve to visualise and document software

[1] From personal experience.

[2] Given that the process is deterministic

[1], while Entity-Relation diagrams focus on the relations between different components, making it ideal for visualising database designs [10].

Different formats exist for storing these diagrams. Examples include XMI - the standard diagram interchange format for UML, most commonly used by the Eclipse Modelling Framework [11], the Rose Petal format - used by the UML development program IBM Rational Rose [12], PlantUML - an open-source textual standard for representing various diagrams including UML and ER diagrams [13], Visual Paradigm files - software that allows for modelling UML, architecture diagrams, business flows etc. [14], and UTML - a program and file format used at the University of Twente for representing UML and various other types of diagrams, its file format building on the JSON standard [15] [16].

The degree to which automated grading is implemented can vary as well. We divide autograding into the following categories: *non-automated* (everything must be done by a human), *automated* (part of the process requires no human input), and (fully) *automatic* (no human input is required). In this paper, we only consider autograders that fall into the categories *automated* and *automatic*.

1.2. Research Questions

In order to examine the feasibility of automatically grading UML diagrams saved in the UTML format, we provide a main research question (**MRQ**):

To what extent can UML diagrams be graded automatically while maintaining or improving the accuracy, consistency, and grading transparency of human grading?

We aim to answer the main research question with the following research questions:

RQ1: To what extent are existing solutions suitable for use in autograding UTML diagrams with regards to (1) accuracy, (2) consistency, (3) grading transparency, (4) extent of linking ILOs to grading instructions, (5) UTML support, and (6) ease of extending the source code to alter functionality?

RQ2: To what extent can a suitable autograder be constructed from previous work to be able to grade UTML diagrams?

RQ3: To what extent does the suitable autograder compare to human grading in the context of grading first-year UML exam questions?

RQ1 is answered in Section 2, by analysing existing work for suitability of grading. These works are categorised according to the requirements outlined by **RQ1** and presented in Table 1. Section 3 explains how we built our autograder, and section? offers perspectives into its performance compared to human grading.

2. RELATED WORK

In order to answer research questions **RQ1**, we conduct a small-scale literature study covering roughly 40 works. It aims to provide an exploratory view into the world of autograders and ILOs, which is why we omit formal inclusion and exclusion criteria. Works are collected using the search engines Google Scholar[3] and ResearchGate[4], with related work for autograders being searched with terms including but not limited to “automatically grading UML diagrams”, “autograder diagram”, “UML diagram assessment”, “machine learning diagrams”, and “diagram evaluation assessment AI”. For papers on ILOs and ILO integration into rubrics, terms were used such as “learning outcomes include in rubric”, “learning objectives in rubrics”, and similar. Snowballing (the practice of looking at sources of sources) was used to a depth of 1.

2.1. Autograders

Multiple types of papers on autogrades are researched, which we categorise into frameworks for autograders, purely algorithmic autograders, and AI-driven autograders (using Machine Learning (ML) / Generative AI (Gen AI) techniques). Findings on autograder implementations are summarised in Table 1.

2.1.1. Frameworks / Theoretical

Autograder frameworks dictate certain designs or methodologies for building autograders. We

[3] https://scholar.google.com
[4] https://www.researchgate.net

present summaries of the explored frameworks and provide a general summary of all frameworks.

N. Smith *et al.* [8] provide a five-step framework for assessing "possibly ill-formed or inaccurate diagrams" that include the steps (1) segmentation, (2) assimilation, (3) identification, (4) aggregation, and (5) interpretation. While the first two steps are meant for translating images or other "raster-based input" into diagrammatic primitives, which is not useful for us, the latter stages provide a solid conceptual foundation to grade diagrams.

F. Batmaz [17] takes a broader look at the process of grading, identifying and developing techniques to reduce repetitive actions, focusing on database ER diagrams. The paper suggests a semi-automatic grading system, including automatic grading based on identifying identical segments between a submission and the solution. Assuming multiple submission revisions are available, it suggests to "not only [use] the reference text but also the intermediate diagrams" for identifying semantic matches [17, p.40]. While multiple solutions are not useful for the purpose of grading only a single submitted diagram after an exam, this might be useful for live feedback.

V. Vachharajani *et al.* [18] propose a UML use case assessment architecture, providing a useful catalogue about edge cases related to use case diagram assessment which are also applicable to other types of diagrams, such as the chance of misspellings, synonyms, abbreviations, directionality of relationships, and more.

W. Bian *et al.* [19] establish a model to map submissions to example solutions and one to grade submissions. It recommends syntactic matching to help with spelling mistakes, semantic matching to match related words, and structural matching to match neighbouring elements and/or inheritance, with the goal to optimally match parts of a student submission with a sample solution.

In conclusion, most autograder strategies recommend structural matching (to identify similar segments of graphs), often in combination with syntactic matching that accounts for misspellings and semantic matching to account for synonyms or related words. Unfortunately, the strategies do not account for integrating ILOs into the grading process explicitly.

2.1.2. Algorithmic

Next to frameworks for autograders, implementations were also discovered during the literature review, of which a subset used purely algorithmic methods. Summaries of these sources are discussed in this section, along with a general summary on algorithmic autograders.

W. Bian *et al.* [5] implements their previous framework [19] and validates it with a case study, using the Levenshtein distance between words for syntactic matching, several metrics for semantic matching, and structural matching based on similar attributes and operations within classes. They find that multiple teacher solutions result in more accurate grades with an average accuracy over 95% [5, p.10]. Additionally, they find that grading configurations need to change per exam if the goal is to produce the most similar scores to manual grading, likely because the focus of each exam or exercise is on a different aspect of diagram creation (associations, inheritance, etc.). Lastly, they state that autograding "has shown to be more consistent and able to ensure fairness in the grading process" compared to manual grading [5, p.11]. Additionally, their visual feedback system seems to provide a clear visualisation of the grading results, which might feel more intuitive for students. An example is shown in Figure 1.

M. Hosseinibaghdadabadi *et al.* [20] also implements the framework by W. Bian *et al.* [19], comparing UML use case diagrams to one or multiple example solutions and preferring the maximum grade. It uses a graph similarity algorithm which matches nodes based on structural matching, along with syntactic matching using the Levenshtein distance between words and semantic matching using WordNet similarity score (HSO, WUP, LIN metrics). It achieves a similarity percentage of 93.31% [20, p.114].

O. Anas *et al.* [21] compares UML class diagram submissions to an example solution. It uses graph similarity scores based on structural matching along with syntactic and semantic matching. Syntactic matching is done with sub-

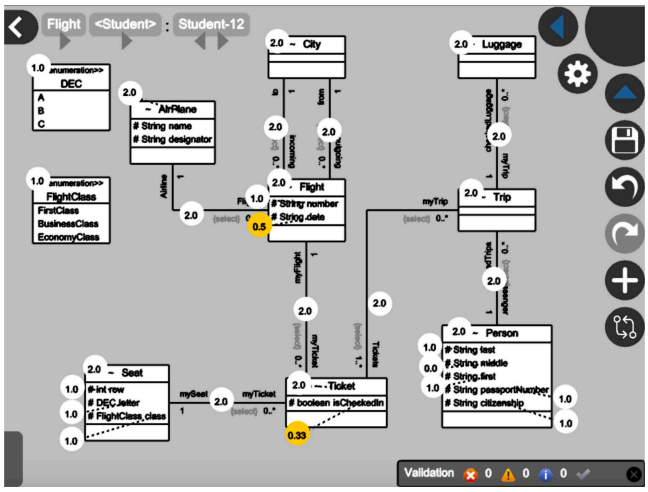

Figure 1: Visual feedback module from W. Bian *et al.* [5, Fig.9]

string matching and semantic matching is done with neighbour similarity ("the comparison of the neighboring classes" [21, p.1585]), relationship name, type, multiplicity, and inheritance. It achieves a respectable correlation with human grading, with more than 80% identical grades, more than 90% achieving a corelation larger than 0.85, and no grading achieving a correlation lower than 70%.

Multiple papers mention the use of XMI [22] [23], the object notation standard by OMG [11], or Rose Petal files [24] [25], the standard of IBM Rational Rose [12], but fail to mention specifics about matching algorithms or results.

H. AlRawashdeh *et al.* [26] provides an interesting alternative way of grading submissions: by means of combining many UML diagram validators, model checkers, and even LTL properties given by instructors. However, a clear purpose, scope, and results are missing from the paper.

M. Striewe *et al.* [27] also attemps property checking akin to H. AlRawashdeh *et al.* [26]'s work by focusing on graph queries for evaluation, providing a domain-specific language that looks similar to SQL. While it offers an interesting alternative approach, the fact that teachers would have to learn a query language and transform their existing rubrics/example solutions into this format could be a real hurdle, especially given the performance of graph-isomorphism-based solutions [5] [20] [21]. Additionally, the paper does not account for misspellings or the use of synonyms.

S. Foss *et al.* provide multiple papers on AutoER, a database diagram generator and evaluator that provides direct interaction with a description text [28] [29] [30]. It is more geared towards interactive use, intended for providing intermediate results and hints during the diagram creation process, before handing in the final submission. Unfortunately, concrete comparisons to manual grading and its source code could not be found.

P. Thomas, like S. Foss *et al.*, also provides a selection of papers on the automatic grading of ER diagrams [9] [31] [32] [33] [34]. Unlike S. Foss *et al.*, these papers are focused on a *single* assessment point and provide a grading strategy that accounts in its basis for imprecise diagrams (diagrams containing misspellings, duplicate entities, etc.). They base their analysis on comparing ever increasing subsets of the graph ((Minimal) Meaningful Units) based on the work of N. Smith *et al.* [8]. By 2009, P. Thomas *et al.* manage to achieve a correlation to human grading of 92%, along with statistically proving that the autograder grades more consistently than human grading. The grading results can be viewed in Figure 2.

In 2011, P. Thomas *et al.* provide an online platform for both students and teachers to ease the process of automatic grading further, also used by N. Smith *et al.* [35], which further mathematically specifies P. Thomas *et al.*'s work. Unfortunately, we were not able to locate the source code of this grader.

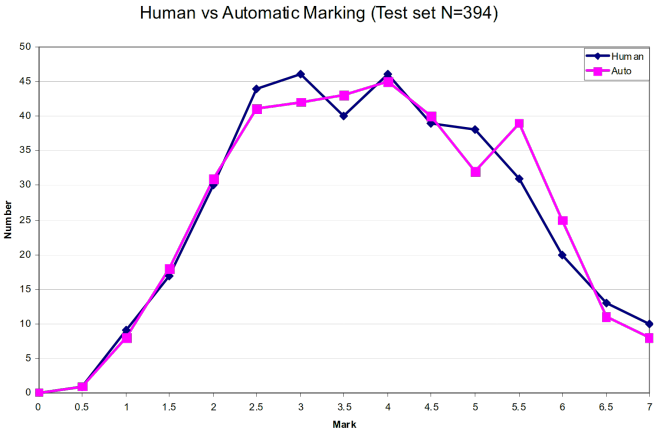

Figure 2: P. Thomas *et al.* [33, Fig. 3]: Human vs. automatic grading in database ER diagrams.

In conclusion, most existing implementations of autograders use some graph isomorphism algorithm with a combination of structural, semantic, and syntactic matching, as suggested by most frameworks. Some solutions attempt to autograde using property or formula checking,

but fail to mention a detailed enough methodology or results to warrant further investigation. No autograders provide methods on integrating ILOs into the grading process.

2.1.3. ML- / Gen AI-driven

Next to using purely algorithmic methods, some papers experiment with Machine Learning / Generative AI (collectively: 'AI-driven solutions') to automatically grade submissions, and even mention some hybrid AI-driven / algorithmic solutions. We provide summaries of the explored sources below, along with a general conclusion on AI-driven autograders.

D. R. Stikkolorum *et al.* [36], one of the earliest found sources, attempts Machine Learning-based autograding using several machine learning algorithms to compare submissions to expert grades. Unfortunately, the grading only reaches a maximum accuracy of 42.76%, while rounding off scores to a 10-point integer scale. Exact methods and algorithms are not mentioned.

C. Wang *et al.* [37] evaluate the feasibility of LLM-based grading with the LLM model ChatGPT-4o, focusing on student reports containing multiple types of UML diagrams. They feed pictures of student-submitted UML diagrams directly into the model along with an explanatory prompt that aims to trigger a Chain-of-Thought process (which should help LLMs "tackle complex arithmetic, commonsense, and symbolic reasoning tasks" [38]), and runs the model once per submission, with a temperature of 0.1. They find that score differences range from −0.25 to +3.75 points, with with the LLM handing out significantly lower average scores compared to humans. Additionally, they note many occurrences of incorrect grading (wrong identifications, overstrictness, and misunderstandings) [37, p.18], which means that, while the authors claim that their solution "demonstrates particular proficiency in the automated evaluation of UML use case diagrams", the grading is not internally consistent and contains hallucinations. In the words of the authors: "In the evaluation based on UC4, GPT deducts points for missing relationships between specified actors and use cases, but theses relationships existed in the UML use case" [37, p.13]. Furthermore, the paper does not express a strong correlation between LLM grading and human grading, at least compared to papers utilising graph matching algorithms [33] [20], nor does it recognise the inherent bias of LLMs [39] or their inherent nondeterminisim (even with a zeroed temperature) [40] [41].

N. Bouali *et al.* [42] uses various LLMs (Llama 3.2B, ChatGPT-o1 mini, and Claude Sonnet) to grade diagrams, first translating the diagrams into text instead of giving the LLM images directly like C. Wang *et al.* [37]. While they achieve a Pearson correlation to human grading of 0.76 with both ChatGPT and Claude, they run into the same inconsistency issues as C. Wang *et al.*: "while the models would provide a final score as requested in the prompt's response format, this score often did not match the actual sum of points awarded in their criterion-by-criterion assessment", and "'One ChargingPort is associated with One Vehicle' was matched with 'One ChargingPort is associated with One ChargingStation' with a similarity of 0.92, despite describing different domain relationships" [42, p.164].

N. Bouali *et al.* identify the problem with grading with LLMs perfectly, stating that "This discrepancy can be attributed to the autoregressive nature of LLMs, where they generate responses token by token" [42, p.164]. Because these models are in their very essence based on predicting tokens [43], there is no formal guarantee that the grades are internally consistent and that grades are produced accurately with respect to the rubric. The fact that LLMs produce grades that correlate with human grading does not mean that this grading is done in a fair, consistent, or reliable manner. While N. Bouali *et al.* try to reduce the nondeterminisim of LLMs by setting the temperature to zero, this does not necessarily remove non-determinism [40] [41], nor does it account for training biases [39], as mentioned before.

R. Ramachandran *et al.* [44], unlike the previous papers, use a human-in-the-loop design in combination with both purely algorithmic steps, using LLMs only for semantic and syntactic matching. Using structural matching algorithms similar to papers presented in Section 2.1.2, it achieves a Mean Average Error of only 0.611,

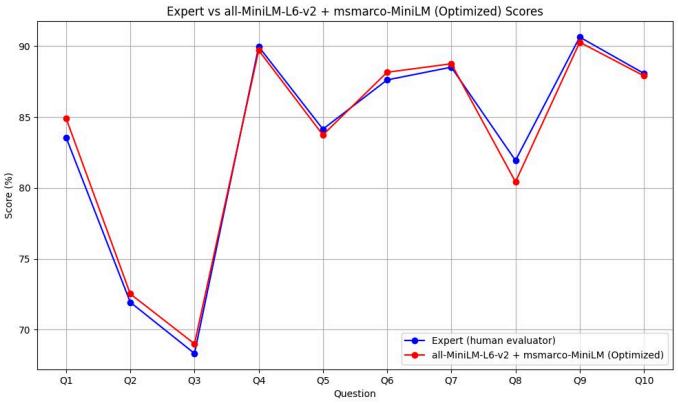

Figure 3: R. Ramachandran *et al.* [44, p.13]: Comparison of expert scores and CodeLLama scores using a combination of `all-MiniLM-L6-v2` and `msmarc-MiniLM` as word similarity models.

aligning very closely to human grading (see Figure 3). Unfortunately, the data set contains only ten self-procured images, which negatively impacts the significance of these results, not to mention that the nondeterminism introduced by the LLMs will impact the consistency of grading, although it is unclear to what extent.

In conclusion, while AI-based grading has been attempted in recent years, purely AI-driven solutions produce lacking similarity to human grading compared to graph isomorphism-based solutions as well as introducing non-deterministic and biased behaviour, while providing no consistency guarantees. This makes these types of solutions inferior to graph isomorphism solutions in terms of *accuracy*, *consistency*, and *grading transparency*. When used only for semantic and/or syntactic matching, it can provide similar accuracy to algorithmic solutions, although it still introduces nondetermism in grading which negatively affects *consistency*.

2.2. Intended Learning Objectives and examination

A. Dinur *et al.* [45] states that analytical rubrics (those which mention explicit criteria) provide more details than global, holistic rubrics. These types of rubrics (and their exercises) can be constructed directly from the ILOs of a course or module, which would provide a detailed grading rubric that aligns closely to its ILOs [4]. Given that rubrics and exercises are defined in such a way, one could link these ILOs to the grading rubric and provide functionality for an autograder to show these in the final grade This would indicate to students how well they achieved the learning goals of the module in order to improve *grading transparency*.

2.3. Conclusion

In the explored related work, existing frameworks primarily recommend structural matching in combination with syntactic and semantic matching to account for spelling mistakes and the use of synonyms. Existing implementations mostly use the methods recommended by the frameworks, with the best results stemming from determinstic graph isomorphism algorithms. Purely AI-driven methods may require less effort from teachers, since teachers do not need to produce sample diagrams but can describe their rubric in words, but produce noticeably inferior results to graph matching algorithms. Using hybrid methods, specifically using AI-driven classification algorithms only for semantic/syntactic matching, seems to produce similar results to 'pure' graph matching algorithms, but does not necessarily provide accuracy gains over algorithmic solutions and can additionally introduce nondeterminism in otherwise determinstic solutions, reducing consistency.

3. SESHAT

In order to automatically grade student submissions, we develop *Seshat* [47]: a generic autograder capable of automatically analysing and grading theoretically any type of diagram. For the purposes of this paper, we offer built-in support for UTML.

Seshat uses the techniques from Section 2 and Table 1 which gave the best results in terms of accuracy, consistency, and grading transparency: a graph isomorphism algorithm for structural matching, Levenshtein distance for syntatic matching, and `all-MiniLM-L6-v2` [48] for semantic matching.

We first tried out Princeton's WordNet [49], as it also performs semantic similarity checks, but these scores did not reflect the expected semantic similarity after testing with several self-synthesised examples and some example synonyms from the datasets. WordNet matches strictly according to the hierarchy of singular words, meaning that examples in the dataset such as

Author	Di	Ac	Co	Tr	F	A	I	R	ILO	UTML
W. Bian *et al.* [5]	UML Class	H	H	H	-	-	i	r	N	N
M. Hosseinibaghdadabadi *et al.* [20]	UML Use Case	H	H	H	-	-	i	r	N	N
O. Anas *et al.* [21]	UML Class	M	H	H	-	-	i	r	N	N
S. Modi *et al.* [22]	UML Class	?	H	H	-	-	-	-	N	N
R. Jebli *et al.* [23]	UML Class	?	H	H	-	-	-	-	N	N
N. H. Ali *et al.* [24] [25]	UML Class	?	H	L	-	-	-	-	N	N
H. AlRawashdeh *et al.* [26]	UML State/Sequence	?	H	?	-	-	I	-	N	N
M. Striewe *et al.* [27]	UML Class	?	H	H	-	-	I	-	N	N
S. Foss *et al.* [28] [29] [30]	ER	?	H	?	-	-	I	r	N	N
P. Thomas *et al.* [9] [31] [32] [33] [34]	ER	H	H	H	-	-	I	r	N	N
D. R. Stikkolorum *et al.* [36]	UML Class	L	L	L	-	-	-	-	N	N
C. Wang *et al.* [37]	UML	M	L	M	F	a	I	r	N	N
N. Bouali *et al.* [42]	UML Class	M	M	M	F	a	I	r	N	N
R. Ramachandran *et al.* [44]	ER	H	M	H	F	a	i	r	N	N

Table 1: Autograders and their suitability scores.

What* **Di*agram types are supported*, *how high is the* **Ac***(curacy)*, **Co***(nsistency)*, and *Grading* **Tr***(ansparency)*, *how* **F***indable,* **A***ccessible,* **I***nteroperable, and* **R***eproduable is the tool*, *can the tool link* **ILO***s to grading*, *and how well is* **UTML** *supported*?

Scoring (except FAIR) is divided into "N" (*No Support*), "L" (*Low*), "M" (*Medium*), "H" (*High*), and "?" (*Unknown*), which gives an indication of each autograder's suitability w.r.t. that particular criterium. The scoring for these rubrics is done in a comparative way, with the lowest-scoring solution receiving a "L" or "N" and the highest scoring receiving a "H". High **accuracy** is awarded for deterministic solutions, with lower values given to nondeterministic programs. High **consistency** is awarded for determinstic solutions. High **grading transparency** is awarded for solutions that explain the final grade in terms of rubrics (medium for full rubrics that might not match (i.e. AI-driven solutions)). **ILO** and **UTML** support is given a "H" or "N" based on inclusion of these features. **FAIR** scoring is done by checking the Findability, Accessibility, Interoperability, and Reusability, inspired by M. D. Wilkinson [46, Box 2, p.4]. We focus purely on the autograder solutions for this rubric. For example, if the code is findable with a fixed ID or link, the project is available but only through a paywall, the algorithms in the paper are designed to be interoperable with only one diagram format (for example XMI) and only one type of diagram (for example ER diagrams), and the work is partially reproducible (deriving parts of the source code using algorithms in the paper), it gets a score of 'Fa_r'.

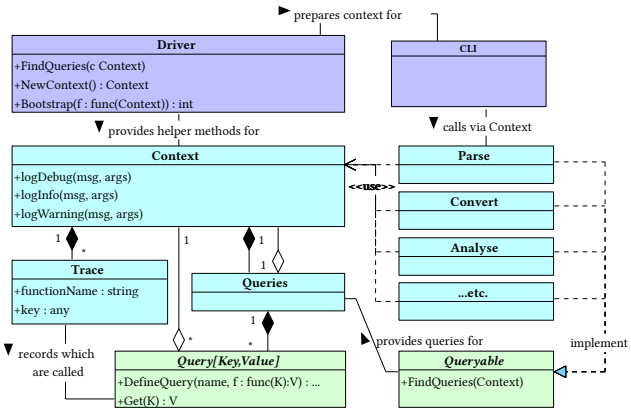

Figure 4: Query architecture for *Seshat*.

'ChargingPort' v.s. 'ChargingStation' would not be matched since 'Port' is in a different WordNet category than 'Station', even though in the exercise they are semantically similar, while examples such as 'Team Member' and 'Virtual Machine' got a semantic match, while sharing no semantic similarity within any of the datasets.

3.1. Architecture and Language

The goal of *Seshat* is to take input (from either exam exports, a list of files, or via some other format), transform the input into an internal graph representation, run comparison algorithms on it defined in Section 2.1.2 which produces a set of scores (a 'grade'), and format this grade in a certain way. The exact methodology, algorithms, and visualisations are likely to change, which is why we aim to maximally decouple these parts.

In order to achieve this, we implement a query-based architecture akin to that of the Rust compiler [50] (an example is given in Figure 4). This encourages decoupling each stage of the process, and additionally increases transparency internally in the grading process, as one can easily query intermediate solutions from the grading process. Testing components is also inherently made easier due to the split-up functionality.

This architecture also allows us to cache all stages of the grading process if they are split up into separate queries, which should allow for some improvements in grading speed. Note that this is only possible because the autograder only contains determinstic algorithms.

The autograder is entirely written in Go [51]. A minimal CLI is included to offer a minimal textual interface and showcase the query architecture.

3.2. Features

This section describes the feature set of *Seshat* by walking the reader through the process of grading a dataset. Features are explained in the order of the grading process.

3.2.1. Parsing

The parsing step transforms a diagram file into an structure that *Seshat* can understand. For UTML, it merely parses the JSON UTML structure and adds some metadata such as the file name.

3.2.2. Transformation into internal representation

In order to be able to grade diagrams, *Seshat* uses an internal representation of a graph. This choice is made to separate the grading from any one specific diagram format, a mistake made in some other autograders (see Table 1). Separating the grading from a diagram format has the benefit of being able to easily integrate new diagram formats into *Seshat* by merely writing a translation from a particular format into an internal representation.

To support as many diagram formats as possible, this internal graph definition is very loosely defined. It contains as little semantic concepts as possible (inheritance, multiplicities, etc.) as it aims to capture the literal shapes and text. This allows *Seshat* to store possibly broken submissions, which allows us to explicitly check the well-formedness of a graph at a defined stage, or ignore certain broken aspects of graphs, which helps in the perceived *leniency* of the autograder and should bring it closer to human grading.

The structure is defined by some metadata such as the filename and a collection of vertices and edges. Each vertex and edge has a unique identifier. Vertices additionally contain a title, some values (fields or methods), certain optional semantic properties such as visibility (public/private/protected/…) and type (Class/Interface/…), and visual properties such as location and size.

Edges are always directed and can connect at either end to a vertex, edge or nothing. Next to an identifier, they have stylistic properties for the starting and ending point of the edge, such as arrow style and text, as well as general stylistic properties, such as the style of the edge (dotted or solid). Finally, each edge has visual properties that define the edge's path. This path can be of any length, allowing for complex paths.

3.2.3. Error correction

After a diagram is converted into *Seshat*'s internal representation, *Seshat* tries to correct specific kinds of mistakes made in the diagram creation process, depending on which options are enabled and how they are configured. This also allows for encoding semantic meaning into the diagram if the original diagram format does not allow for expressing certain semantics.

For example, since UTML does not allow edge-to-edge connections, the 'reparation' phase can correct for this, given that edges are sufficiently visually close. These error-correcting and/or semantic encoding features exist to allow maximum leniency in grading and exist on the internal representation level, meaning they automatically apply to all diagram formats *Seshat* supports.

For visualising error corrections, we refer to submission 154286, shown in Figure 5, as well as its corrected version, shown in Figure 6.

3.2.3.1. Edge Label Swapping

Firstly, *Seshat* supports edge label swapping. It can happen during the diagram creation process that a student creates labels on an edge, but then drags the labels to other locations. This can be seen with the edge 'Project' → 'Deliverable', which has swapped labels 'has ->' and '0..1', in Figure 5 and Listing 1.

Having swapped labels is a problem for automatic grading, because *Seshat* does not consider the visual representation of the submission. When a student drags around labels and the diagram tooling does not automatically swap the labels internally, the file structure does not match the visual structure, which would make *Seshat* grade a diagram 'incorrectly' according to student and teacher expectations, putting students at an unfair disadvantage.

Seshat includes an option `swap_edge_labels` for this, which looks for labels with large offsets towards another position on the edge and swaps them if possible. The distance at which this occurs is configurable as a percentage of the length of an edge.

3.2.3.2. Edge 'Anchoring'

Some diagram software, such as UTML, sadly does not allow connecting edges to other edges, which is mandatory syntax for concepts such as UML association classes. Alternatively, students may drag arrows close to vertices or edges but not connect them directly for several possible reasons, including time limitations of exams.

This is a problem for autograders, since *technically*, those edges are not connected to anything, meaning that students will get points deducted for a solution that looks visually correct.

In order to allow some level of leniency which compensates for the inability of students to connect edges and/or the diagram software's inability to support it, *Seshat* allows for 'anchoring' disconnected edges that are sufficiently close to another vertex or edge. The distance at which anchoring occurs can be configured as a percentage of the length of an edge. for vertices, this is the length of the top, left, right, or bottom part of the vertex (which is assumed to be a rectangle). A demonstration of this reparation can be seen in the connections 'Record' and 'Internal Information' in Figure 6.

3.2.3.3. Directed Edge Recombination

In diagrams, it is not uncommon to have a set of multiple vertices connect to one 'main' vertex. This happens especially often when drawing multiple inheritance classes in UML. However, diagram software might not make it easy to draw these in a visually pleasing manner, or students might choose to draw separate edges which capture the semantics visually, but which is not represented in the underlying diagram format. A good example of this is presented in Figure 5, where classes ranging from 'SoftwareLicense' until 'CloudService' connect to 'Resources'. However, these edges are drawn individually as straight edges, and any given edge does not connect an inheriting class to the inherited class.

This is a major problem for autograding, as the representation mismatch will cause autograders to grade the individual edges, instead of the semantic concept (for example, multiple-inheritance).

Seshat includes an option `simplify-directed-edges` for this, which looks for multiple edge-to-edge connections that end in a directed edge (an edge with some kind of arrow or diamond on one side). The effect of this recombination is demonstrated in Figure 6.

3.2.4. Grading process

After the internal representation is repaired, it can be graded. This is done based on graph comparisons / isomorphism: a teacher solution is compared to a student solution by trying to make a mapping of teacher vertices and edges to those drawn by the student. *Seshat* grades with the following general graph comparison algorithm:

1. take the teacher's graph (r) and a student submission (s). Vertices in a graph g are denoted by V_g. Edges in g are denoted by E_g.
2. analyse the semantic and syntactic equivalence of all $(v_r, v_s), v_r \in V_r, v_s \in V_s$ and score it in a range of 0 (no similarity) to 1 (perfect similarity), i.e. score$(v_r, v_s) \rightarrow [0..1]$.
3. take max(score(v_r, v_s)) $\forall v_r \in V_r, v_s \in V_s$, given that the score is higher than the threshold (defined in the grading instructions). Put these pairs $v_r \rightarrow v_s$ into a minimal subgraph pair (g_{v_r}, g_{v_s})
4. repeat for each pair g_{v_r}, g_{v_s} until no progress is made:
   1. get a list of $e_r \in E_r$ that connect to $v_r \in g_{v_r}$ ($E_{g_{v_r}}$), and a list of $e_s \in E_s$ that connect to $v_s \in g_{v_s}$ ($E_{g_{v_s}}$).
   2. $\forall e_r \in E_r$ get the starting and ending vertices of e_r (if they exist), collect them into $V_{g_{v_r}}$. Do the same for E_s ($V_{g_{v_s}}$).
      - make a mapping of max(score$(v_r, v_s)), v_r \in V_{g_{v_r}}, v_s \in V_{g_{v_s}}$ into `newFixedIds`.
   3. Add all $(v_r, v_s) \in$ `newFixedIds` and add all $e_r \in E_r, e_s \in E_s$ given that their starting/ending vertices are in V_r or V_s respectively.

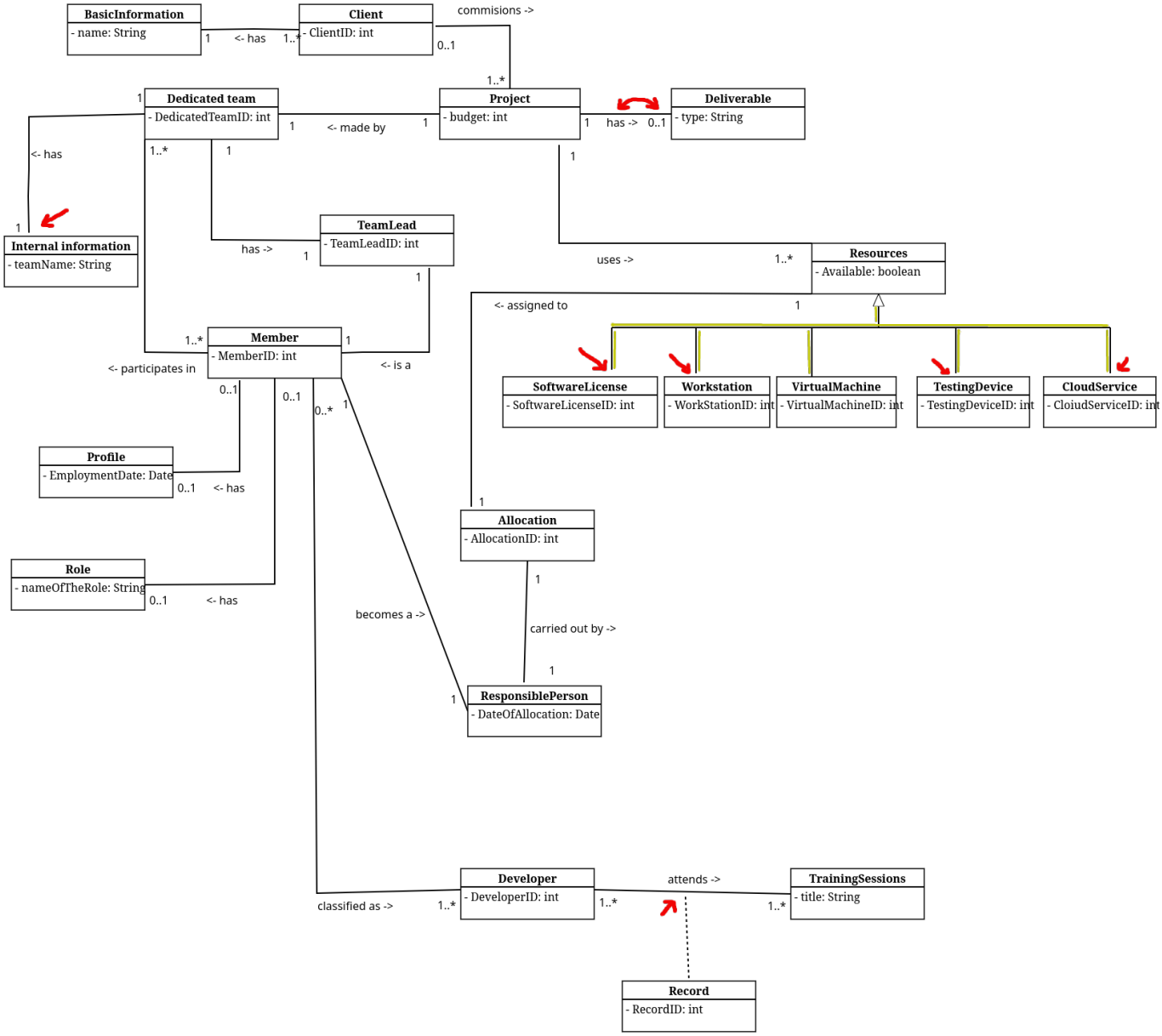

Figure 5: A student submission [47, 2025_M2_BIT/q/1/154286.json] containing several inconsistencies (marked red and yellow): 'Internal Information' has a disconnected edge. The edge between 'Project' and 'Deliverable' has internally swapped labels (see Listing 1). The edge of 'Record' is not connected to the 'attends ->' edge between 'Developer' and 'TrainingSession'. The inherited classes of 'Resources' are all separate edges, and most edges are not connected to the inheriting classes.

```
...
"middleLabel": { // middleLabel denotes the center of the edge
  "offset": { //...but the offset places it at the location of 'endLabel'
    "x": 45,
    "y": -6
  },
  "edgeLocation": 1,
  "value": "0..1"
},
  "endLabel": { // endLabel denotes the end of the edge
    "offset": { //...but the offset places it at the location of 'middleLabel'
      "x": -70,
      "y": -6
    },
    "edgeLocation": 2,
    "value": "has ->"
  },
  ...
```

Listing 1: 154286.json lines 96-111, showing internally flipped labels. From the graph of Figure 5.

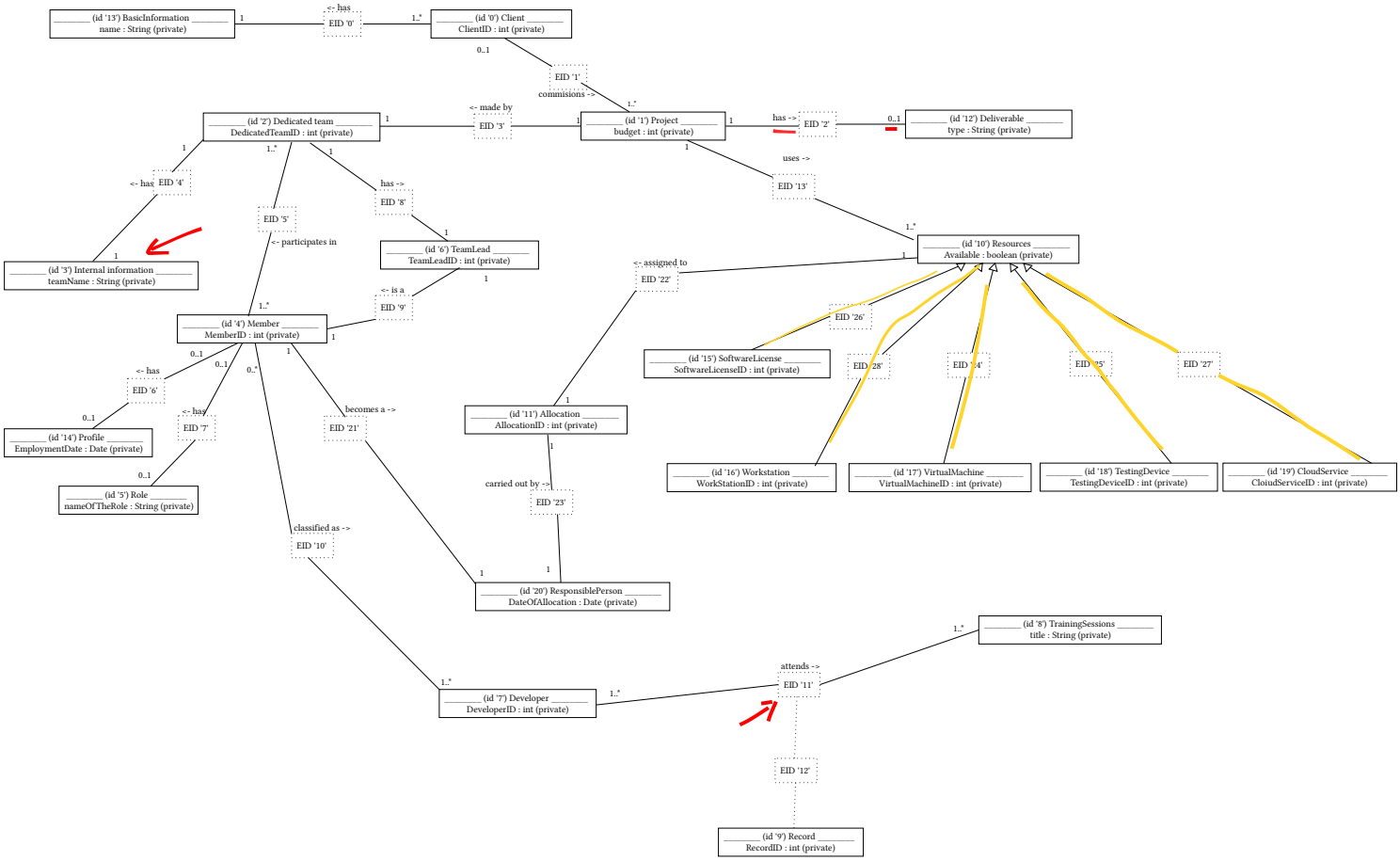

Figure 6: A GraphViz view of the corrected internal representation of submission 154286. Note that due to the limitations of GraphViz we show edges as two edges with a fake node inbetween. Edge paths are correctly stored internally.

5. Once no further progress is made, we score the subgraphs $\left(g_{v_r}, g_{v_s}\right)$ based on how many vertices and edges have been mapped.
6. We take the highest-scoring subgraph pair as basis and add as many other matching subgraphs, given that the edges and vertices of other subgraph pairs do not appear in the final combined solution yet.

After the last step, we have one final mapping $\left(g_{v_r}, g_{v_s}\right)_{\text{fin}}$ which decides which vertices of the reference solution likely map to the student submission.

After arriving on a final mapping, we apply the user-supplied grading configuration: we hand out additions or deductions in points based on vertex or edge presence, absence, or incorrectness, and for specific parts of vertices or edges such as vertex attributes or edge arrow style.

This grading configuration also specifies the reparation options specified in Section 3.2.3. This allows a teacher / grader to specify stricter or looser corrections for a particular dataset, if *Seshat* performs undesired repairs.

3.2.5. Visualisation

After arriving on a grade, *Seshat* needs to show this to the user. Two built-in methods exist: an export to a `.csv` file, containing only the final grades per input file, and an export to `.json` files, one per input file, explaining in detail how *Seshat* calculated the grade.

Add example .csv / .json export

There is also a demo graph export of a grade, which shows in red, yellow, and green which vertices / edges were incorrect, superfluous, or correct.

add example graded graph

There also exists a GraphViz export function for *Seshat*'s internal structure. This allows for easily viewing how different reparation options affect a solution. A JSON export of the graph is also possible, which outputs the exact internal representation of the graph. An example is given in Figure 6. Note that GraphViz neither supports fixed edge paths nor edge-to-edge connections, meaning that edge placement is incorrect, however, this is present in the JSON export.

3.3. Testing

When developing an autograder, it is vitally important to verify its correctness and expected behaviour. We implement a variety of automated tests to ensure this. Because *Seshat* includes a lot of parsing and graph corrections, not unlike compilers and Syntax Tree / Control Flow Graph analysis, we take inspiration from the terms used for compiler testing, as mentioned in V. Zaytsev [52].

Seshat is tested in an automated fashion for all large features it possesses: for parsing, conversion into the internal graph structure, repairing the internal graph, and for grading.

For the initial stage of parsing UTML into our own in-memory data structure, we employ P-testing [52]. This means that we verify that, for each file in our test data the data sets, *Seshat* should produce the exact same JSON structure as is inputted. One noteable exception is UTML's `attributes` and `methods` fields. the UTML files sometimes either lack these fields, they are `null`, or they contain an an empty array (`[]`). Because this is semantically equivalent in our context, we explicitly treat `null`/`[]` or a missing `attributes`/`methods` field as the same.

For the conversion from UTML into the internal representation, we use a form of N-testing [52]: we parse a UTML file into `ParseResultUTML`, then convert it into our `InternalGraph`, and then perform checks comparing the parse result and internal representation. We validate whether the vertex and edge IDs remain the same, and whether edges are still connected to their respective vertices or edges, to name a few.

For special features such as connecting edge ends and swapping labels (see Section 3.2.3) we perform unit testing with fixed examples, which test both a couple of happy paths (where the program should modify the graph) and control paths (where the program should not change the graph).

4. THREATS TO VALIDIDTY

The internal validity of this paper may be affected by a few factors.

Firstly, selection bias in the researched papers and reporting bias in those papers may affect the perceived performance of certain types of autograders, which could negatively affect the choice of algorithms used for *Seshat*. As we aim to provide a *comprehensive* overview, not an exhaustive one, selection bias becomes more important, especially when using snowballing, as one can easily fall into multiple papers about a specific sub-strategy and lose the overall picture. This is also why we keep snowballing at a minimum.

Secondly, the manner in which humans have constructed the rubrics in the dataset is inherently biased towards human graders, which generally do not apply it to the letter (which we will see in Section 5). When implementing a grading rubric that *does* follows the rubric to a tee, we risk losing the leniency of human interpretation.

Thirdly, the datasets do not provide details about which humans graded which submissions, when those submissions were graded, which reasoning was behind each of the grades, along with a variety of other factors. Especially the reasoning behind a grade would have been useful to discern the different types of grading deductions, which would help with result quality. However, since we only get a final grade to work with, we have to guess why grades were constructed by humans, and aim to achieve a similar leniency and error correction. This process is additionally prone to bias from the author.

5. RESULTS

/discussion?

5.1. BIT 2024

results

5.2. TCS 2025

5.2.1. Question 5

- To compare against M2_2025_TCS, I will likely have to adjust the grading to not penalise extra classes and/or fields, just purely give points for the things that are present, like specified in the rubric.

5.2.2. Question 6

results

5.3. BIT 2025

results

6. DISCUSSION

add

7. CONCLUSION

add

BIBLIOGRAPHY

[1] Object Management Group, "OMG website." [Online]. Available: https://www.omg.org/

[2] F. Ahmed *et al.*, "Teaching Assistants as Assessors: An Experience Based Narrative," 2024. [Online]. Available: https://research.utwente.nl/files/457355611/126242.pdf

[3] M. Meadows *et al.*, "A Review Of THe Literature On Marking Reliability." [Online]. Available: https://assets.publishing.service.gov.uk/media/5a820a57e5274a2e87dc0d5a/0505_Meadows_and_Billington_CERP_RP.pdf

[4] D. Osinga, "Combining Dynamic and Static Analysis for the Automation of Grading Programming Exams," Bachelor thesis, 2024. [Online]. Available: https://github.com/osingaatje/ut-bachelor-thesis

[5] W. Bian *et al.*, "Is automated grading of models effective?: assessing automated grading of class diagrams," in *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems*, in MODELS '20. ACM, Oct. 2020, pp. 365–376. doi: 10.1145/3365438.3410944.

[6] I. G. Pirie, *The measurement of programming ability*. University of Glasgow (United Kingdom), 1975. [Online]. Available: https://search.proquest.com/openview/3afb41488c34283f38dec7f13a3ea744/1

[7] M. J. Rees, "Automatic assessment aids for Pascal programs," *ACM SIGPLAN Notices*, vol. 17, no. 10, pp. 33–42, Oct. 1982, doi: 10.1145/948086.948088.

[8] N. Smith *et al.*, "Interpreting imprecise diagrams," in *Diagrammatic Representation and Inference*, in International Conference on Theory and Application of Diagrams. Mar. 2004, pp. 239–241. doi: 10.1007/978-3-540-25931-2_24.

[9] P. Thomas, "Grading Diagrams Automatically," 2004. [Online]. Available: https://oro.open.ac.uk/90155/1/2004_01.pdf

[10] S. Bagui *et al.*, *Database design using entity-relationship diagrams*. Auerbach Publications, 2003. [Online]. Available: https://www.taylorfrancis.com/books/mono/10.1201/9780203486054/database-design-using-entity-relationship-diagrams-sikha-bagui-richard-earp

[11] OMG, "XML Metadata Interchange format." [Online]. Available: https://www.omg.org/spec/XMI

[12] IBM, "Rational Rose." [Online]. Available: https://www.ibm.com/support/pages/ibm-rational-rose-enterprise-7004-fix-pack-4-7000

[13] PlantUML, "PlantUML." [Online]. Available: https://plantuml.com/

[14] Visual Paradigm, "Visual Paradigm." [Online]. Available: https://www.visual-paradigm.com/

[15] D. Huistra, "UTML - internal GitLab repository." [Online]. Available: https://gitlab.utwente.nl/ewi/eduapps/UTML/

[16] University of Twente, "utml.apps.utwente.nl." [Online]. Available: https://utml.apps.utwente.nl/

[17] F. Batmaz, "Semi-Automatic Assessment ofStudents' Graph-Based Diagrams," 2010. [Online]. Available: https://www.academia.edu/download/66135135/70_22270_EM_26aug_20feb_L.pdf

[18] V. Vachharajani *et al.*, "A Proposed Architecture for Automated Assessment of Use Case Diagrams," in *International Journal of Computer Applications (0975 – 8887)*, 2014. [Online]. Available: https://www.academia.edu/download/67672696/pxc3900193.pdf

[19] W. Bian *et al.*, "Automated Grading of Class Diagrams," in *2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, IEEE, Sept. 2019, pp. 700–709. doi: 10.1109/models-c.2019.00106.

[20] M. Hosseinibaghdadabadi *et al.*, "Automated Grading of Use Cases," in *2023 ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems (MODELS)*, IEEE, 2023. [Online]. Available: https://ieeexplore.ieee.org/iel7/10343461/10343549/10343598.pdf

[21] O. Anas *et al.*, "New method for summative evaluation of UML class diagrams based on graph similarities," 2021. [Online]. Available: https://www.academia.edu/download/66135135/70_22270_EM_26aug_20feb_L.pdf

[22] S. Modi *et al.*, "A Tool to Automate Student UML diagram Evaluation," 2021. [Online]. Available: https://www.academia.edu/download/72488756/575.pdf

[23] R. Jebli *et al.*, "Assessing Students' UML Class Diagrams: a NewAutomated Solution," in *2023 7th IEEE Congress on Information Science and Technology (CiSt)* |, IEEE, 2023. [Online]. Available: https://ieeexplore.ieee.org/iel7/10409867/10409868/10409936.pdf

[24] N. H. Ali *et al.*, "Assessment System For UML Class DiagramUsing Notations Extraction," 2007. [Online]. Available: https://www.researchgate.net/profile/Zarina-Shukur/publication/253243639_Assessment_System_For_UML_Class_Diagram_Using_Notations_Extraction/links/55487af30cf2b0cf7acec2e4/Assessment-System-For-UML-Class-Diagram-Using-Notations-Extraction.pdf

[25] N. H. Ali *et al.*, "A Design of an Assessment System for UML Class Diagram," in *2007 International Conference on Computational Science and its Applications (ICCSA 2007)*, IEEE, Aug. 2007, pp. 539–546. doi: 10.1109/iccsa.2007.2.

[26] H. AlRawashdeh *et al.*, "Using Model Checking Approach for Grading the Semantics of UML Models," 2014. [Online]. Available: https://iieng.org/images/proceedings_pdf/8684E0114567.pdf

[27] M. Striewe *et al.*, "Automated Checks on UML Diagrams," in *ITiCSE'11*, in ITiCSE '11. ACM, June 2011, pp. 38–42. doi: 10.1145/1999747.1999761.

[28] S. Foss *et al.*, "Learning UML database design and modeling with AutoER," in *Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings*, in MODELS '22. ACM, Oct. 2022, pp. 42–45. doi: 10.1145/3550356.3559091.

[29] S. Foss *et al.*, "Automatic Generation and Marking of UML Database Design Diagrams," in *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education*, in SIGCSE 2022. ACM, Feb. 2022, pp. 626–632. doi: 10.1145/3478431.3499376.

[30] S. Foss, "AutoER: A System for the Automatic Generation and Evaluation of UML Database Design Diagrams," 2022. [Online]. Available: https://open.library.ubc.ca/media/download/pdf/24/1.0421624/4

[31] P. Thomas *et al.*, "An approach to the automatic grading of imprecise diagrams," technical report, 2006. doi: .org/10.21954/ou.ro.00016046.

[32] P. Thomas *et al.*, "Automatically assessing graph-based diagrams," *Learning, Media and Technology*, vol. 33, no. 3, pp. 249–267, 2008, doi: 10.1080/17439880802324251.

[33] P. Thomas *et al.*, "Automatically Assessing Diagrams," in *Proceedings of the IADIS International Conference on e-Learning*, 2009. [Online]. Available: https://www.researchgate.net/profile/Pete-Thomas/publication/42799920_Automatically_assessing_diagrams/links/0fcfd5060076dd8ba2000000/Automatically-assessing-diagrams.pdf

[34] P. Thomas *et al.*, "Generalised Diagramming Tools with Automatic Marking," in *Innovation in Teaching and Learning in Information and Computer Sciences*, 2011. doi: 10.11120/ital.2011.10010022.

[35] N. Smith *et al.*, "Automatic Grading of Free-Form Diagrams with Label Hypernymy," in *2013 Learning and Teaching in Computing and Engineering*, IEEE, Mar. 2013, pp. 136–142. doi: 10.1109/latice.2013.33.

[36] D. R. Stikkolorum *et al.*, "Towards Automated Grading of UML Class Diagrams with Machine Learning," 2019. [Online]. Available: https://ceur-ws.org/Vol-2491/paper80.pdf

[37] C. Wang *et al.*, "Assessing UML Diagrams by GPT: Implications for Education," technical report, 2025. [Online]. Available: https://www.researchgate.net/publication/397720325_Assessing_UML_Diagrams_by_GPT_Implications_for_Education

[38] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," 2023. [Online]. Available: https://arxiv.org/abs/2201.11903

[39] R. Ranjan *et al.*, "A Comprehensive Survey of Bias in LLMs: Current Landscape and Future Directions," 2024. doi: 10.48550/arXiv.2409.16430.

[40] M. Brenndoerfer, "Why Temperature=0 Doesn't Guarantee Determinism in LLMs," 2025, [Online]. Available: https://mbrenndoerfer.com/writing/why-llms-are-not-deterministic

[41] B. Atil *et al.*, "Non-Determinism of "Deterministic" LLM Settings," 2025. [Online]. Available: https://arxiv.org/pdf/2408.04667

[42] N. Bouali *et al.*, "Toward Automated UML Diagram Assessment: Comparing LLM-Generated Scores with Teaching Assistants," 2025. [Online]. Available: https://research.utwente.nl/files/496461589/134819.pdf

[43] A. F. Ferraris *et al.*, "The architecture of language: Understanding the mechanics behind LLMs," *Cambridge Forum on AI: Law and Governance*, vol. 1, pp. 1–19, 2025, doi: 10.1017/cfl.2024.16.

[44] R. Ramachandran *et al.*, "AI Assisted System for Automated Evaluation of Entity-Relationship Diagram and Schema Diagram Using Large Language Models," technical report, Dec. 2025. doi: 10.3390/bdcc10010002.

[45] A. Dinur *et al.*, "Incorporating outcomes assessment and rubrics into case instruction," *Journal of Behavioral and Applied Management*, vol. 10, no. 2, pp. 291–311, 2009, [Online]. Available: https://jbam.scholasticahq.com/article/17258.pdf

[46] M. D. Wilkinson, "The FAIR Guiding Principles for scientific datamanagement and stewardship," 2016. [Online]. Available: https://www.nature.com/articles/sdata201618

[47] D. Osinga, "Seshat." [Online]. Available: https://codeberg.org/drwr/seshat

[48] Hugging Face, "sentence-transformers/all-MiniLM-L6-v2." [Online]. Available: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

[49] Princeton University, "WordNet - A Lexical Database for English." [Online]. Available: https://wordnet.princeton.edu/

[50] Rust Team, "Overview of the Rust Compiler." [Online]. Available: https://rustc-dev-guide.rust-lang.org/overview.html

[51] Google, "The Go Programming Language." [Online]. Available: https://go.dev/

[52] V. Zaytsev, "An Industrial Case Study in Compiler Testing (Tool Demo)," 2018. doi: 10.1145/3276604.3276619.